



Polimorfismo

O que é Polimorfismo?

- Polimorfismo é um dos pilares da Programação Orientada a Objetos (POO), junto com:
 - Encapsulamento
 - Herança
 - Abstração
- A palavra vem do grego: "poli" = muitos, "morfos" = formas
→ Significa que objetos de diferentes classes podem responder de maneira diferente ao mesmo método.

Conceito Geral e Exemplos do Mundo Real

- O polimorfismo permite que um mesmo método tenha comportamentos diferentes, dependendo do objeto que o chama.
- Exemplos:
 - `fazerSom()`
 - Um cachorro late
 - Um gato mia
 - Um pato grasna
 - Botão "Imprimir":
 - Imprime documentos PDF, imagens ou e-mails → mesma ação, diferentes comportamentos

Por que Polimorfismo é importante?

- **Desacopla** código cliente das implementações concretas
- **Reutiliza** código com abstrações genéricas (interfaces, superclasses)
- **Facilita extensões** por meio de subclasses
- **Permite substituição** de objetos sem alterar o código principal

✨ Com o polimorfismo, é possível escrever **código mais genérico, reutilizável e fácil de manter.**

Tipos de Polimorfismo em Java

- Em Java, o polimorfismo se manifesta em dois tipos principais:
 - Polimorfismo de tempo de compilação/estático ou sobrecarga de métodos (*method overloading*)
 - Polimorfismo de tempo de execução/dinâmico ou sobrescrita de métodos (*method overriding*)

Tipo de Polimorfismo	Quando ocorre	Mecanismo
Compilação (Estático)	Durante a compilação	Sobrecarga (<i>overload</i>) de métodos
Execução (Dinâmico)	Durante a execução	Sobrescrita (<i>override</i>) de métodos

Polimorfismo de Tempo de Compilação (Estático)

- Também chamado de sobrecarga de métodos (*method overloading*).
- Permite que uma classe tenha múltiplos métodos com o mesmo nome, mas com diferentes assinaturas (número ou tipo de parâmetros).
- A decisão de qual método será executado ocorre em tempo de compilação.
- Exemplo em Java:

```
somar(int a, int b)  
somar(double a, double b)
```

Ambos têm o mesmo nome, mas tipos de parâmetro diferentes.

Regras da Sobrecarga em Java

- Mesmo nome de método
- Assinaturas diferentes:
 - Número de parâmetros
 - Tipos de parâmetros
 - Ordem dos parâmetros
- O tipo de retorno não conta para diferenciar métodos sobrecarregados
- Os métodos podem coexistir na mesma classe

Exemplo de Sobrecarga

```
public class Impressora {  
    public void imprimir(String texto) {  
        System.out.println("Texto: " + texto);  
    }  
  
    public void imprimir(int numero) {  
        System.out.println("Número: " + numero);  
    }  
  
    public void imprimir(String texto, int vezes) {  
        for (int i = 0; i < vezes; i++) {  
            System.out.println("Repetição " + (i + 1) + ": " + texto);  
        }  
    }  
}
```

Impressora

- + imprimir(texto : String) : void
- + imprimir(numero : int) : void
- + imprimir(texto : String, vezes : int) : void

Polimorfismo de Tempo de Execução (Dinâmico)

- Também chamado de sobrescrita de métodos (*method overriding*).
- Ocorre quando uma subclasse redefine um método da superclasse com a mesma assinatura.
- Permite alterar ou especializar o comportamento herdado.
- A decisão sobre qual método será executado é feita em tempo de execução, com base no tipo real do objeto.

Regras da Sobrescrita em Java

- A subclasse deve manter a **mesma assinatura** do método (nome, tipo e ordem dos parâmetros).
- O **tipo de retorno** deve ser o mesmo ou um subtipo do método da superclasse.
- O método original **não pode ser** `private` nem `final`
- O **nível de acesso** na subclasse pode ser igual ou mais permissivo.
- Recomenda-se o uso da anotação `@Override`

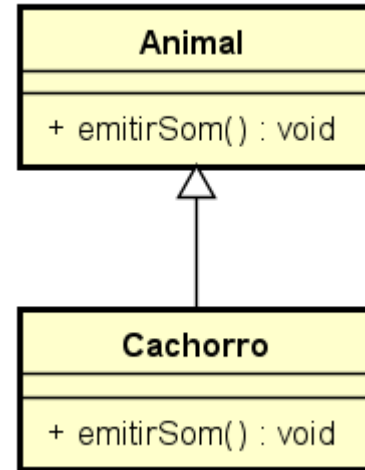
A Anotação @Override

- @Override informa ao compilador que estamos sobrescrevendo um método herdado.
- Caso a assinatura não corresponda exatamente ao método original, o compilador gerará um erro.
- Garante maior segurança, clareza e legibilidade no código.

Exemplo de Sobrescrita

```
public class Animal {  
    public void emitirSom() {  
        System.out.println("Som do animal:");  
    }  
}
```

```
public class Cachorro extends Animal {  
    @Override  
    public void emitirSom() {  
        System.out.println("Au au!");  
    }  
}
```



- Cachorro redefine o comportamento de `emitirSom()`, herdado de `Animal`.
- Mesmo que a referência seja da superclasse, o comportamento da subclasse será executado.
 - Permite que cada subclasse forneça **sua própria versão especializada** de um comportamento comum.

O que é `super` em Java?

- A palavra-chave `super` permite acessar métodos e atributos da superclasse.
- É usada em subclasses para:
 - Chamar métodos sobrescritos da superclasse;
 - Acessar construtores da superclasse;
 - Complementar comportamentos herdados, sem substituí-los por completo.

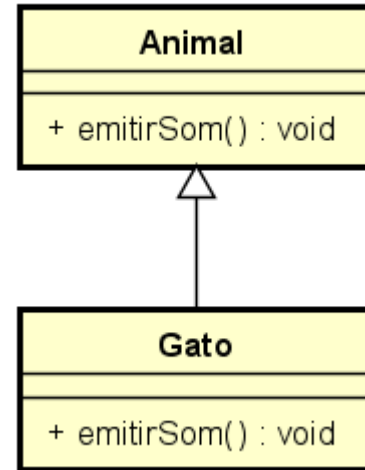
Quando usar `super` em métodos?

- Quando queremos executar a lógica original da superclasse antes, depois ou no meio de um novo comportamento:
 - Reutilizar código já existente na superclasse;
 - Evitar duplicações de lógica;
 - Adicionar funcionalidades sem descartar as anteriores;
- Permite criar versões mais especializadas sem perder a funcionalidade herdada.

Exemplo de Sobrescrita com `super`

```
public class Animal {  
    public void emitirSom() {  
        System.out.println("Som do animal:");  
    }  
}
```

```
public class Gato extends Animal {  
    @Override  
    public void emitirSom() {  
        super.emitirSom(); // Chama o método da superclasse  
        System.out.println("Miau!");  
    }  
}
```



- `super.emitirSom()` → permite que o Gato mantenha o comportamento do `Animal`, mas acrescente sua própria identidade sonora.

Referência de Superclasse em Java (1/2)

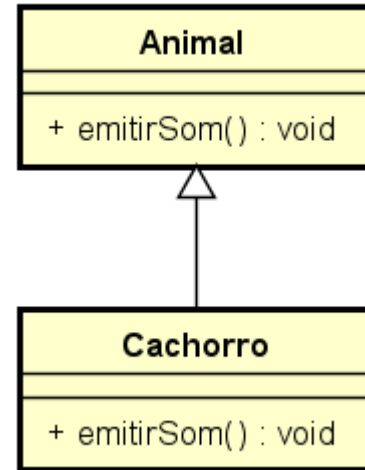
- Em Java, é possível usar uma variável de referência da superclasse para apontar para um objeto de uma subclasse.
- Essa é uma das principais formas de aplicar o polimorfismo em tempo de execução.
- Com isso, podemos escrever código mais genérico, reutilizável e flexível.

Referência de Superclasse em Java (2/2)

- Uma variável do tipo da superclasse pode armazenar um objeto de qualquer **subclasse**.
- Quando chamamos um **método sobrescrito**, a versão da **subclasse** é executada — não a da superclasse.
- Isso é possível graças à **vinculação dinâmica (*dynamic binding*)**, tratada mais adiante.

Exemplo de Referência de Superclasse em Java

```
public class Animal {  
    public void emitirSom() {  
        System.out.println("Som do animal:");  
    }  
}  
  
public class Cachorro extends Animal {  
    @Override  
    public void emitirSom() {  
        System.out.println("Au au!");  
    }  
}  
  
public class Teste {  
    public static void main(String[] args) {  
        Animal a = new Cachorro(); // referência de superclasse  
        a.emitirSom(); // imprime: "Au au!" - método da subclasse  
    }  
}
```



Mesmo sendo `a` do tipo `Animal`, o método sobrescrito em `Cachorro` é o que será executado.

O que são Classes Abstratas?

- **Classe abstrata**: não pode ser instanciada diretamente.
- Serve como **base para outras classes**.
- Pode conter:
 - **Métodos com implementação** (comportamento padrão);
 - **Métodos abstratos** (sem corpo), que exigem implementação nas subclasses.
- Utiliza-se o modificador `abstract`

O que são Métodos Abstratos?

- **Método abstrato:**
 - Declarado **sem corpo**.
 - Deve ser obrigatoriamente implementado pelas subclasses concretas.
- **Somente classes abstratas podem ter métodos abstratos.**
- Exemplo de sintaxe:

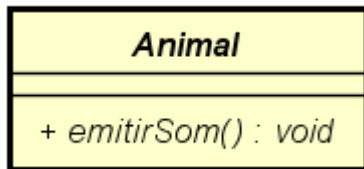
```
public abstract class Animal {  
    public abstract void emitirSom(); // sem corpo  
}
```

Para que Servem os Métodos e Classes Abstratos?

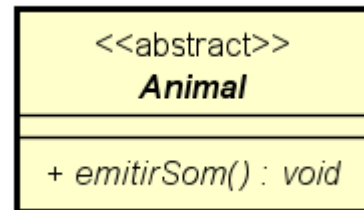
- Evitar instanciamento de classes genéricas ou incompletas.
- Forçar subclasses a implementar certos métodos.
- Promover polimorfismo em níveis superiores da hierarquia de classes.
- Aumentar consistência e previsibilidade na arquitetura do sistema.

Representação em UML

- Classes abstratas aparecem com o nome em itálico.
- Métodos abstratos também aparecem em itálico.
- Também é comum o uso do estereótipo «abstract»
- Isso indica que a classe não pode ser instanciada diretamente.



ou



Estereótipos em UML

- Estereótipos adicionam significado semântico aos elementos do diagrama.
- São palavras entre « » que classificam o elemento.
- Exemplos:
 - «abstract» → classe abstrata
 - «interface» → interface
 - «enumeration» → enumeração
 - «singleton» → padrão de projeto
- Tornam o diagrama mais expressivo e compreensível.

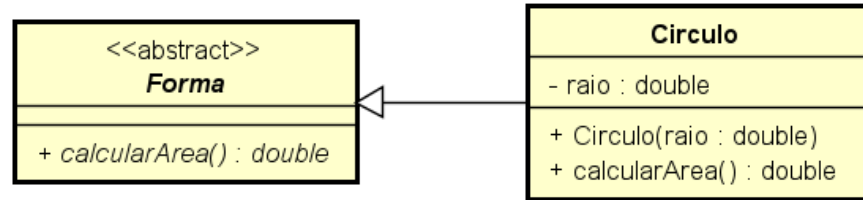
Exemplo de Classes e Métodos Abstratos

```
public abstract class Forma {
    public abstract double calcularArea();
}
```

```
public class Circulo extends Forma {
    private double raio;

    public Circulo(double raio) {
        this.raio = raio;
    }

    @Override
    public double calcularArea() {
        return Math.PI * raio * raio;
    }
}
```



```
public class TesteForma {
    public static void main(String[] args) {
        Forma f = new Circulo(3.0);
        System.out.println("Área: " + f.calcularArea());
    }
}
```

Polimorfismo em *Arrays*

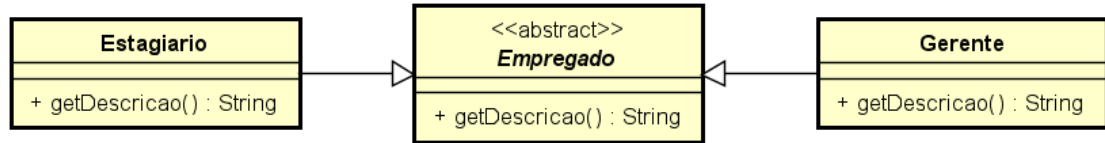
- Com polimorfismo, podemos:
 - Tratar objetos de subclasses diferentes por meio de referências da superclasse.
 - Criar arrays de objetos heterogêneos (desde que todos sejam instâncias da mesma hierarquia).
- Arrays polimórficos:
 - Um *array* de referências da superclasse pode conter objetos de diversas subclasses.
 - O método chamado em tempo de execução será o da classe real do objeto, não da variável de referência.

Exemplo de Array Polimórfico

```
public abstract class Empregado {
    public String getDescricao() {
        return "Empregado comum";
    }
}
```

```
public class Gerente extends Empregado {
    @Override
    public String getDescricao() {
        return "Gerente da empresa";
    }
}
```

```
public class Estagiario extends Empregado {
    @Override
    public String getDescricao() {
        return "Estagiário temporário";
    }
}
```



```
public class TesteArray {
    public static void main(String[] args) {
        Empregado[] equipe = new Empregado[3];
        equipe[0] = new Gerente();
        equipe[1] = new Estagiario();
        equipe[2] = new Estagiario();

        for (Empregado e : equipe) {
            System.out.println(e.getDescricao());
        }
    }
}
```

A saída do programa será diferente para cada posição do *array*, mesmo que todas as variáveis sejam do tipo *Empregado*.

Polimorfismo em Parâmetros de Método

- Métodos podem receber parâmetros do tipo da superclasse.
- É possível passar qualquer instância de subclasse como argumento.
- Isso simplifica o código e aumenta a reutilização.

```
public class Relatorio {  
    public void imprimirDescricao(Empregado e) {  
        System.out.println("Descrição: " + e.getDescricao());  
    }  
  
    public static void main(String[] args) {  
        Relatorio rel = new Relatorio();  
        rel.imprimirDescricao(new Gerente());  
        rel.imprimirDescricao(new Estagiario());  
    }  
}
```

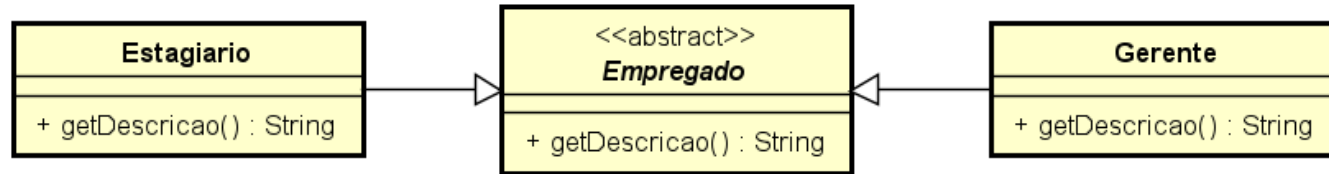
O método `imprimirDescricao()` trata diferentes objetos de forma genérica, mas executa comportamentos específicos de cada subclasse.

Polimorfismo e Projeto Orientado a Boas Práticas

- O polimorfismo melhora:
 - Modularidade
 - Flexibilidade
 - Manutenibilidade
- Mas deve ser usado com boas práticas de projeto para garantir código limpo e sustentável.

Polimorfismo e Projeto Orientado a Boas Práticas

- Evite acoplamento com tipos concretos:



// Menos recomendado:

```
public void processarFuncionario(Gerente g) {
    System.out.println(g.getDescricao());
}
```

// Recomendado:

```
public void processarFuncionario(Empregado e) {
    System.out.println(e.getDescricao());
}
```

Polimorfismo e Projeto Orientado a Boas Práticas

- Extensibilidade e o Princípio Aberto/Fechado (*Open/Closed Principle*):

“Entidades de software devem ser abertas para extensão, mas fechadas para modificação.”

- Isso significa:
 - Novas funcionalidades podem ser adicionadas via subclasses;
 - Sem necessidade de modificar código já testado;
 - Redução de bugs em sistemas evolutivos.

Polimorfismo e Projeto Orientado a Boas Práticas

- Evite hierarquias confusas:
 - Use herança apenas quando “X é um Y”.
 - Para “X tem um Y”, prefira composição ou agregação.
- Princípio de Substituição de Liskov (LSP):

“Objetos da superclasse devem poder ser substituídos por objetos das subclasses sem alterar o comportamento correto do programa.”

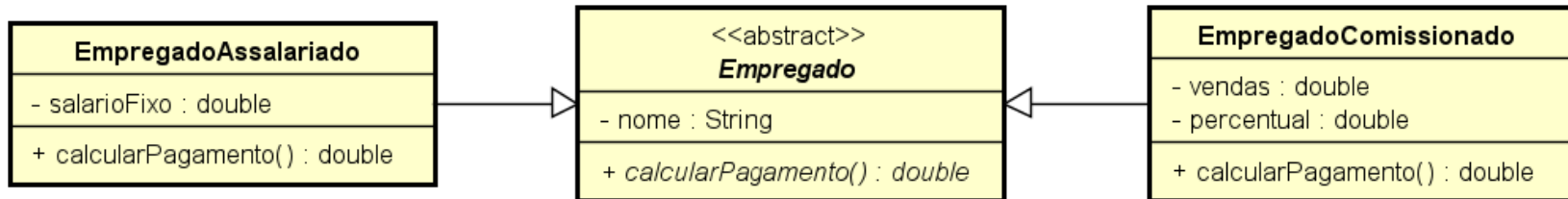
- Subclasses devem respeitar o contrato da superclasse!



Barbara Liskov

Exemplo Completo de Polimorfismo em Java

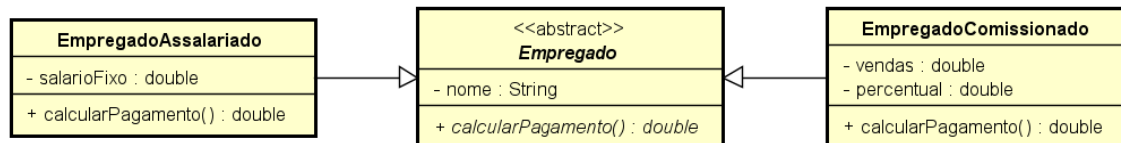
- **Classe abstrata** `Empregado` com o **método abstrato** `calcularPagamento()`
- **Subclasses:**
 - `EmpregadoAssalariado` (pagamento fixo)
 - `EmpregadoComissionado` (pagamento por comissão)
- **Pontos abordados:**
 - Sobrescrita de métodos (`@Override`)
 - Uso de *arrays* com referência à superclasse
 - Vinculação dinâmica



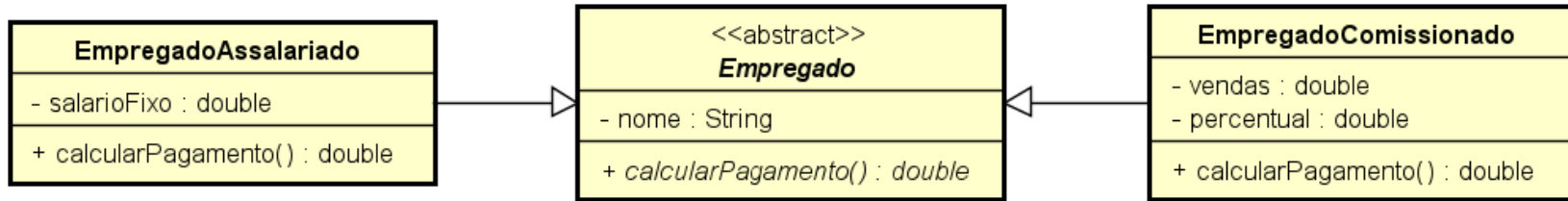
Exemplo Completo de Polimorfismo em Java

```
public abstract class Empregado {
    private String nome;
    public Empregado(String nome) { this.nome = nome; }
    public String getNome() { return nome; }
    public abstract double calcularPagamento();
}
```

```
public class EmpregadoAssalariado extends Empregado {
    private double salarioFixo;
    public EmpregadoAssalariado(String nome, double salarioFixo) {
        super(nome);
        this.salarioFixo = salarioFixo;
    }
    @Override
    public double calcularPagamento() {
        return salarioFixo;
    }
}
```



Exemplo Completo de Polimorfismo em Java



```

Empregado[] lista = new Empregado[3];
lista[0] = new EmpregadoAssalariado("João", 3000);
lista[1] = new EmpregadoComissionado("Maria", 50000, 0.1);
lista[2] = new EmpregadoAssalariado("Carlos", 4500);

for (Empregado emp : lista) {
    System.out.println(emp.getNome() + " - Pagamento: R$ " + emp.calcularPagamento());
}
    
```

- O método `calcularPagamento()` é chamado conforme o **tipo real do objeto**

Recapitulando Polimorfismo em Java (1/2)

- **Polimorfismo** é um dos pilares da Programação Orientada a Objetos.
- Esse conceito permite que objetos de diferentes classes sejam tratados de forma uniforme, promovendo:
 - **Extensibilidade** do sistema: novos comportamentos podem ser adicionados sem modificar os existentes.
 - **Reutilização de código**: métodos genéricos podem trabalhar com diferentes tipos de objetos.
 - **Facilidade de manutenção**: alterações em subclasses não afetam o código da superclasse.

Recapitulando Polimorfismo em Java (1/2)

- Dois tipos principais:
 - Polimorfismo de tempo de compilação (sobrecarga de métodos).
 - Polimorfismo de tempo de execução (sobrescrita de métodos).
- Uso da anotação `@Override` para indicar sobrescrita.
- Utilização da palavra-chave `super` para acessar métodos da superclasse.
- Declaração de variáveis com o tipo da superclasse para armazenar objetos de subclasses.
- Compreensão da vinculação dinâmica (*dynamic binding*).
- Aplicação de polimorfismo em arrays e parâmetros de métodos.
- Importância das classes abstratas como base para hierarquias polimórficas.

Exercícios propostos

Verificar atividade no Moodle